Leo Xie

Research Proposal

*SPAR: **S**ynthetic **P**lay from **A**rchived **R**eplays*

SPAR is a general framework for learning how multiple robots should act around each other from passive video of real interactions, combined with synthetic data from simulation (Fig. 2). Given footage of a shared space, a description of that space and its objective, and an outcome for each recorded episode, SPAR rebuilds what every agent did and checks it against the outcome. It then replays real episodes in simulation, multiplies them into synthetic ones, trains multi-robot policies, and runs many training runs that search for strategies that win and differ from anything in the video. Rendered video with exact labels lets SPAR measure the error of its own video-mined data. We test the framework on Override, the current VEX Robotics Competition game, where streamed matches, exact rules and public scores make every stage measurable, and where our own robots and scrimmages give real ground truth and a real deployment. We target the *Autonomous Robots* topical collection Synthetic Data for Robot Learning, which closes on February 1, 2027.

Problem

Robots increasingly work in spaces shared with other agents, such as warehouse fleets, vehicles at intersections, and teams of robots that cooperate and compete. Learning to act well there needs data on how many agents move and respond to each other, and that is the hardest kind of data to collect. Instrumented fleets, motion capture and test vehicles are expensive, and the data they produce is usually private. Even in sports, the best strategy models, such as TacticAI [1], depend on tracking data owned by clubs.

At the same time, large amounts of passive video of multi-agent interaction already exist, from traffic and warehouse cameras to drone surveys, sports broadcasts and robot competition livestreams. The highD dataset [2] showed that one overhead view can yield trajectories for 110,000 vehicles. Most of this video is never used for robot learning because it is only pixels. It is seen from one viewpoint, agents hide behind each other and look alike, and nothing records what each agent did or why.

Simulation has the opposite weakness. It gives unlimited, perfectly labeled experience, but agents trained against simulated partners learn habits real agents do not share [3] and exploit errors in the simulator. Waymax [4], a driving simulator built on real logs, found that RL policies overfit to the simulated agents they trained with.

We ask a general question: can passive, partial video of real multi-agent interaction be combined with synthetic data to train multi-robot policies that work on real robots, and to discover strategies beyond those seen in the video? Answering it means solving three problems that apply to any such setting. First, video has to become usable data, meaning world-frame positions, actions and task state for every agent across a whole episode. Second, the error of that data has to be measured, even though passive video has no ground truth, because data with unknown error is hard to trust for training. Past auto-labeling studies could only measure error where human labels already existed [5], and rendered worlds have been used to test trackers [6], but not to put an error bar on the multi-agent behavior data a learner actually uses. Third, learning has to go beyond copying. Real data alone can only reproduce existing behavior, so new strategies have to come from simulation. A strategy that wins in simulation may only be exploiting the simulator's errors, so proving that it is new and good takes evidence from outside the simulator. Studies of AlphaZero and of professional Go have measured how new AI play is compared with human play [7, 8], but those games come with complete records, while video of robots gives only a partial one.

We test SPAR on Override, the 2026 to 2027 VEX Robotics Competition game (Fig. 1). Two alliances score pins and cups on nine goals, flip four Toggles that decide who owns the yellow pin halves in each quadrant, and race to park in the Midfield during the last ten seconds [9]. Adding to or taking from an opposing alliance's goal is a major violation, so each robot works with its partner on its own goals and competes for the neutral goals, the Toggles and the Midfield. Override has every part of the general problem in a controlled form. Many events stream their matches, often from one fixed camera per field, the field geometry and scoring rules are exact, the official score of every match is public on RobotEvents [10], and the game changes every year. Unlike sports or traffic, the learned policies can also run on real robots. The framework will be tested on in-house robots, which can film its own scrimmages with ground truth, and can use an Isaac Lab model of the Override field built for a related project.

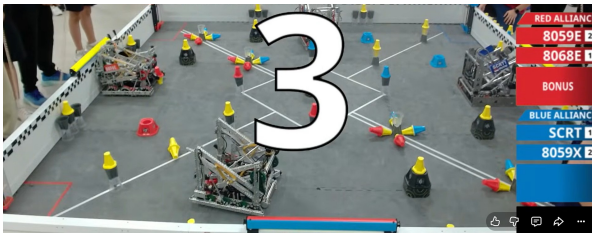


Figure 1: A frame from a VEX event livestream, the kind of passive video our test case starts from. One fixed camera, a countdown and alliance overlay covering the field, and robots that look alike.

Proposed approach

SPAR takes three inputs and runs one loop (Fig. 2). The inputs are passive video of a shared space, a scene spec, and an outcome record for each episode. The scene spec holds everything specific to one setting: the environment map and its landmarks, the agent and object classes, the limits of the agents we control, an objective function that scores a state, and episode timing. Table 1 shows how these inputs map to Override and to two settings that fit the same framework but that we do not test. Everything else is

shared code.

Table 1: SPAR's inputs in general, in our Override test, and in two settings shown only to illustrate the framework.

Input	In general	VEX Override (our test)	Warehouse fleet (not tested)	Road intersection (not tested)
Passive video	Any fixed or slowly moving camera	Event livestreams	Ceiling cameras	Traffic or drone cameras
Scene spec	Map and landmarks, agent and object classes, agent limits, objective, timing	Field tiles and walls, robots, cups, pins and Toggles, our robots' limits, scoring rules, match timing	Floor plan, robots and people, totes, throughput	Lane map, vehicles and pedestrians, travel time and safety, signal timing
Outcome record	A measured result per episode	Official score on RobotEvents, autonomous winner on the stream overlay	Orders completed per shift	Vehicle counts and travel times
Real ground truth	Episodes of agents we control, recorded with an outside tracker	Our scrimmages, filmed like a livestream and tracked overhead, plus robot logs	Our robots' fleet logs	A test vehicle's logs
Simulators	A fast model for search and a detailed one for control	A plan-level simulator for search, an Isaac Lab field model for robot policies	Warehouse simulator	Traffic simulator
Deployment	Policies run on real agents	Our VEX U robots	Fleet robots	A test vehicle

First, SPAR cuts the video into episodes using the outcome record's timestamps, and keeps only segments with a full view of one field, since some streams cut between fields or show replays. It labels agents and objects with foundation models, Grounding DINO [11] with SAM 2 [12], or SAM 3 [13] as an alternative. A smaller detector is then trained on those labels plus rendered frames, so it is fast enough to run over thousands of episodes. In Override an episode is a match, and events.vex gives its start time and the teams on each alliance.

Second, every detection is mapped to world coordinates with a homography fit to the landmarks in the scene spec, once per camera and smoothed over time [14], and detections are linked over time by motion [15]. Each track point stores position, heading and velocity with an uncertainty, plus a tag that says whether it was detected, filled in between detections, or rendered. Identity is a closed set whenever the outcome record lists the agents. In V5RC an alliance is two separate teams, and in VEX U it is one team's 15 in and 24 in robots, so the tracker tells partners apart by motion and, in VEX U, by size.

Third, SPAR rebuilds the task state over time and scores it with the objective function from the scene spec. Comparing that score with the recorded outcome flags bad episodes without human labels. This check is necessary but not sufficient. Errors can cancel inside a total, and swapping two partner robots never changes it, so a pass means the final state is plausible, not that every track is right. We measure how sensitive the check is by injecting known errors into rendered matches, such as an identity swap, a missed pin half or a wrong Toggle, and counting how many it catches. We also report pass rates by match type, since keeping only passing matches could favor quiet ones. In Override the state is checked at the end of the match against the RobotEvents total, and at the end of autonomous against the winner shown on the stream overlay, because RobotEvents publishes only totals.

Fourth, SPAR extracts behavior. Motion comes from the tracks, and discrete events, such as a pin placed or a Toggle flipped, come from rules on object changes next to an agent. These rules are later replaced by an event model in the style of VPT [16], trained on our scrimmage footage synced with robot logs and overhead tracking. Each episode then gets a strategy code, a short readable record of what an alliance tried to do. In Override the code has five fields: the autonomous route, the ordered goals each robot scored on, the share of time the alliance held each Toggle, how each robot split its time among scoring, loading, defense and Toggles, and when each robot entered the Midfield. The distance between two codes adds up the differences field by field, using edit distance for goal orders and absolute differences for shares and times. Two codes count as different strategies only when their distance is larger than the 95th percentile of the distance between two human coders' codes for the same match, measured on 50 matches. On those 50 matches we also report agreement between the coders, and between the coders and the pipeline, with Cohen's kappa [17].

Fifth, SPAR learns in simulation. Real episode states seed the simulator [18], and synthetic episodes multiply them through what-if branches from real moments, perturbed replays [19] and a generative trajectory model [20]. Two simulators share the scene spec. A fast plan-level simulator, with action times fitted from event durations mined from video, runs the large search. The Isaac Lab model trains the policy that runs on our robots, since stacking cups and pins involves too much contact to simulate quickly. Policies are first cloned from real behavior and conditioned on the strategy code, much as AlphaStar conditioned its policy on a statistic drawn from human games [21]. They are then improved with multi-agent RL while a KL penalty keeps them close to real behavior [22]. Scoring rules live in the scene spec, so a rule change creates a new scene spec version. Each match is tagged with the version it was played under, and training and search rerun under the new rules.

Sixth, SPAR searches for strategies beyond the footage. Real play only sets the starting point. The search launches many training runs, each with different settings: the weight of the KL penalty, from close to real play to free of it, the weights on parts of the objective such as Toggles or the Midfield, the mix of opponents, the robot limits it assumes, and the random seed. Different settings push runs toward different strategies, as reward randomization [23] and population based training [24] have shown. An archive keeps the best strategy found in each region of strategy code space [25], and runs earn a bonus for reaching empty regions, so the search

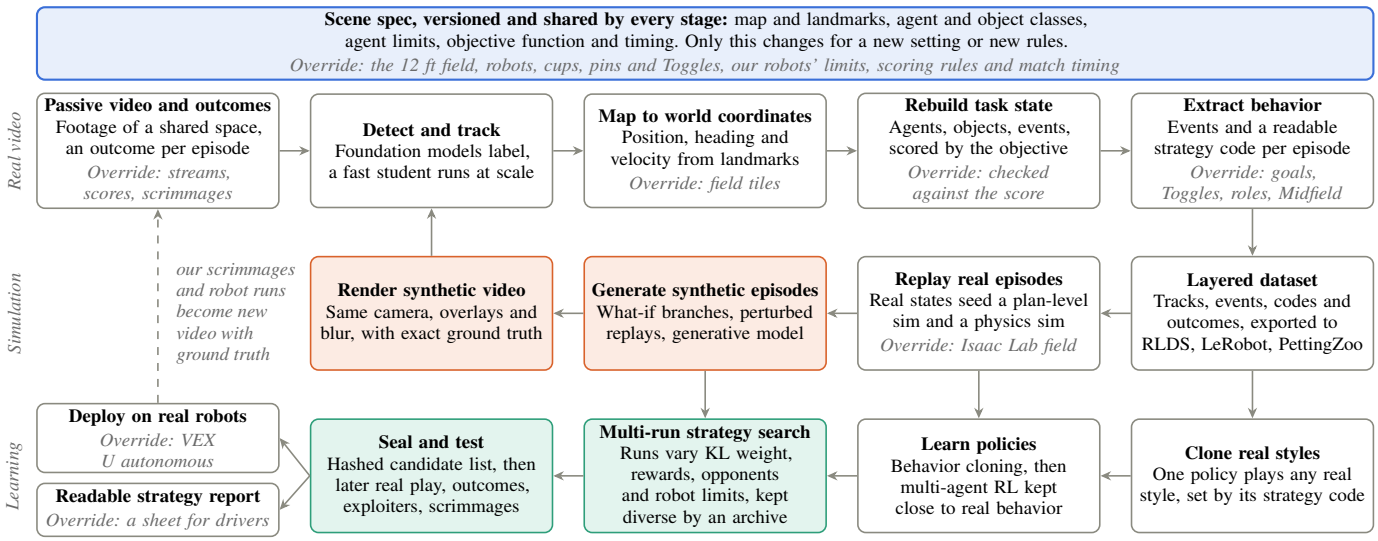


Figure 2: SPAR framework. The scene spec holds everything specific to one setting, and the grey lines show our Override test case. Real video flows along the top row and becomes a layered dataset. The simulator replays and multiplies real episodes and renders synthetic video with exact ground truth for perception. The bottom row learns policies, runs many search runs for new strategies, and tests a sealed list of candidates. Its results go to real robots and to people. Orange boxes produce synthetic data and green boxes carry out strategy discovery.

spreads out instead of piling onto one strong style. A candidate is a strategy code that beats the cloned real styles in simulation and lies farther than the coder threshold from every real style. The best candidates are then attacked by exploiter agents trained only to beat them [21]. Robot limits come from the scene spec, so the search only proposes what our robots can do.

Synthetic data does three jobs in this loop. For perception, rendered video [26] with the same camera view, overlays and blur trains the detector, following the domain randomization recipe [27, 28]. For measurement, every agent in a render was placed on purpose, so renders give exact ground truth for the pipeline's error, which real video cannot. To check that render error carries over to real footage, we film our own scrimmages from a raised side angle like a livestream, while an overhead camera tracks AprilTags [29] on every robot. This gives real multi-robot ground truth with real occlusions. Our drivers can also play assigned strategies, which gives ground truth for the strategy code itself. For learning, synthetic episodes seeded from real ones give scale, variety and realistic opponents and partners, and the whole strategy search runs on synthetic play.

Pre-registration changes how we prove discovery, not how discovery works. Before we process any footage recorded after a cutoff date, we freeze the strategy code, its distance and threshold, and the ranked candidate list, and publish a timestamped SHA-256 hash of that file. We then check whether candidates show up in later real play more often than random codes or codes from self-play under the rules alone, in the spirit of the time split used for Go [8]. We check whether alliances that later used a candidate won more often than a score-only team rating predicts [30], how each candidate holds up against its exploiters, and how it does when our drivers play it against our usual strategies in scrimmages. A candidate that never appears later can still be new and good. The time test only confirms the ones other teams also found, and the other checks cover the rest. For Override the cutoff is November 2026, and we compare against events through January 2027. We also run the same test on the finished Push Back season, where our team already has an RL simulator, and label it as retrospective because we have watched that season.

SPAR writes every episode in a layered format so that other pipelines can use it. The layers are the scene spec version, tracks for every agent and object with their uncertainty and tags, events, the objective state over time, the strategy code, and the outcome. Converters export it to RLDS [31], LeRobot [32], PettingZoo [33] and OG-MARL [34]. Actions recovered from video are marked as inferred, so a learner can weight them differently from logged actions. To test generality, we run the perception and state stages on Push Back and on RECF Pinnacle [35], a 2026 to 2027 game from the REC Foundation that uses the same pins and cups with different scoring. Push Back changes the whole game while Pinnacle changes mostly the objective, and we report the size of each scene spec and the hours needed to adapt.

Expected contributions

We expect five contributions, with the second and fourth being the most new.

- SPAR, a general framework that turns passive, partial video of multi-agent interaction into world-frame tracks, events and strategy codes, set up for each setting by a versioned scene spec and an outcome record, with a layered dataset format and converters to common RL formats.
- Synthetic video as a ground truth oracle whose error predicts error on real footage. Rendered worlds have been used to test trackers [6]. We carry the idea to the quantities a multi-robot learner uses, such as events, task state and strategy codes, and check it against real ground truth from our own scrimmages.
- Strategy-level synthetic data for multi-robot learning, in which real episodes are replayed in simulation and multiplied into synthetic

ones, with a study of how much real and synthetic data each step needs.
- Strategy discovery that goes beyond the footage, through many training runs with different settings and a diversity archive over readable strategy codes, with a pre-registered test against later real play, real outcomes, exploiters and scrimmages. Earlier novelty studies compared AI and human play in games with complete records [7, 8], and we bring this test to multi-robot strategies mined from partial video.
- An evaluation on VEX Override with real robots, generality checks on Push Back and RECF Pinnacle, and a public release of annotations keyed to video and match IDs.

Current approaches and how SPAR differs

Today the parts of this problem are solved separately, each for one setting, and we found no framework that goes from passive video of a shared space to multi-robot policies on real robots with synthetic data in the loop.

Multi-agent behavior data is mostly collected with sensors placed on purpose. Driving research records trajectories from instrumented vehicles or dedicated drone flights, as in highD [2], and builds simulators that replay those logs with simulated agents, such as Waymax [4], along with benchmarks for how realistic those agents are [36]. Sports vision rebuilds player positions from broadcast video [37], and sports strategy work models team tactics [1, 38], mostly from private tracking data. None of these learns strategy from public video and tests it on robots. Rendered worlds have been used to measure tracker error [6], and strategy novelty has been measured in chess and Go [7, 8], but both ideas rely on settings with complete records.

Robot learning from video and synthetic data has grown fast, through video pretraining, real-to-sim replay and generated demonstrations [16, 19, 39–41], but almost all of it is single agent. AlphaStar [21] is the strongest example of strategy discovery, but it relied on 971,000 replays with every action recorded and a perfect simulator, namely the game itself. Within robot competitions, published data use mostly stops at scores. VEX teams scout by hand, VEX Match Jumper [42] only links match records to stream timestamps, FRC analytics [30, 43, 44] work from scores, and hobby projects that track FRC robots in video [45] use the paths for scouting, not learning. The one VEX robot learning paper we found [46] uses no match data.

Table 2: How SPAR compares with the closest prior work.

Work	Learns from public video	Learns team strategy	Synthetic data in learning	Measures its own data error	Discovers new strategies	Runs on a real robot
highD [2]	No, own drone flights	No, trajectories only	No	Yes, against hand labels	No	No
Waymax [4]	No, sensor logs	No	Simulated agents	Not needed, sensor logs	No	No
Virtual KITTI [6]	No, rendered video	No	Rendered video	Yes, for trackers	No	No
SoccerNet-GSR [37]	Yes	No, positions only	No	Yes, against hand labels	No	No
TacticAI [1]	No, private tracking	Yes	Generates setups	Expert ratings only	Suggests corner setups	No
TacSIm [38]	Yes	Yes	Simulated environment	Unclear	No	No
Vid2Player3D [39]	Yes	No, single player	Physics simulation	No	No	No
GR00T N1, DreamGen [40, 41]	Partly, human video	No	Yes	No	No	Yes
AlphaStar [21]	No, game replays	Yes	Self-play league	Not needed, replays are exact	Yes, not measured	No
Schut et al. [7]	No, self-play games	Chess concepts	Self-play	Not needed, games are exact	Yes, against human games	No
Mellmann et al. [47]	No, own cameras	No	No	Yes, motion capture	No	No learning
SPAR (ours)	**Yes**	**Yes**	**Yes**	**Yes, renders checked on real scrimmages**	**Yes, pre-registered test**	**Yes**

The closest prior work is highD and Waymax for turning recorded multi-agent behavior into simulation, Vid2Player3D for going from broadcast video to a simulated policy, and AlphaStar for strategy discovery (Table 2). SPAR joins these into one framework for physical robots, measures the error of its own video-mined data, and produces readable strategies that can be tested for novelty and run on hardware.

Benchmarks

No benchmark exists for learning multi-robot strategy from passive video, so we compare each stage with the strongest available method for that stage, and the whole system with our team's current RL pipeline and autonomous routine (Table 3). We also reuse good public metrics at our scale, such as the GS-HOTA score of SoccerNet-GSR [37] with a tolerance in inches instead of 5 m, and the sim agent realism metrics of the Waymo Open Sim Agents Challenge [36].

Renders give exact ground truth at any scale but are not real, while scrimmage ground truth is real but small. Showing that error measured on renders predicts error on real scrimmage footage is what supports a trustworthy error budget for passive video in general, not only for Override. Before collecting data we count streamed Override matches by program and report how many survive

Table 3: Baselines, metrics and reference data for each stage of SPAR, with Override as the test case.

Stage	Systems and baselines we compare against	Metric	Reference data
Detection	Our current Grounding DINO plus SAM 2 chain [48], SAM 3 zero-shot and with 10 labeled examples per class [13], YOLO-World [49], YOLO fine-tuned on the hand labels	mAP50	About 200 hand-labeled Override frames, plus renders
Tracking and identity	ByteTrack [50], SAMURAI [51], Deep-EIoU [15], CAMELTrack [52]	HOTA, ID swaps per episode	Hand-labeled clips, renders, scrimmages
World coordinates	Per-frame calibration [53], line-based calibration [54], calibration smoothed over time [14]	Position, heading and velocity error	Overhead AprilTag tracks from our scrimmages, robot logs with VEX GPS
Video to task state	The modular design of SoccerNet-GSR [37]	GS-HOTA style score, score check pass rate and share of injected errors caught	Renders, scrimmages, official Override scores
Strategy code	Two human coders	Cohen's kappa, code distance between coders and pipeline	50 hand-coded matches, scrimmages with assigned strategies
Simulated agent realism	Log replay with scripted agents, as in Waymax [4]	Sim agent realism metrics [36], score error of replayed real matches	Held-out Override matches
Policy learning	Our team's MARL pipeline from scratch, behavior cloning only, offline RL from replays [55], MA-GAIL [56], cloning plus RL kept close to real behavior [22]	Win rate against held-out real styles, sample efficiency, outcome prediction against a score-only team rating	Held-out Override matches
Strategy discovery	Plain self-play, an AlphaStar-style league [21], one run without the diversity archive, random codes	Distinct strategies found, share of sealed candidates seen later, win rate beyond team rating, win rate after exploiter attack	Override events after the cutoff, Push Back (retrospective), scrimmages
Hardware	Our team's current autonomous routine	Task score and success rate in the 30 s VEX U autonomous period [9], at least 20 trials per condition	Real Override field
Generality	The same pipeline on Override	Stage metrics, scene spec size, hours to adapt	Push Back and Pinnacle streams

each stage, with V5RC and VEX U reported separately. Every learning result uses at least five seeds with bootstrap 95% confidence intervals, and hardware success rates come with Wilson intervals [57], since 20 trials only pin a rate down to about 20 points either way.

References

[1] Z. Wang, P. Veličković, D. Hennes et al., "TacticAI: An AI assistant for football tactics," *Nat. Commun.*, vol. 15, art. no. 1906, 2024.

[2] R. Krajewski, J. Bock, L. Kloeker, and L. Eckstein, "The highD dataset: A drone dataset of naturalistic vehicle trajectories on German highways for validation of highly automated driving systems," in *Proc. IEEE ITSC*, 2018.

[3] M. Carroll, R. Shah, M. K. Ho et al., "On the utility of learning about humans for human-AI coordination," in *Proc. NeurIPS*, 2019.

[4] C. Gulino, J. Fu, W. Luo et al., "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2023.

[5] B. A. Griffin, M. Gangwar, J. Sela et al., "Auto-labeling data for object detection," arXiv:2506.02359, 2025.

[6] A. Gaidon, Q. Wang, Y. Cabon, and E. Vig, "Virtual worlds as proxy for multi-object tracking analysis," in *Proc. CVPR*, 2016.

[7] L. Schut, N. Tomašev, T. McGrath et al., "Bridging the human-AI knowledge gap through concept discovery and transfer in AlphaZero," *Proc. Natl. Acad. Sci. USA*, vol. 122, no. 13, 2025.

[8] M. Shin, J. Kim, B. van Opheusden, and T. L. Griffiths, "Superhuman artificial intelligence can improve human decision-making by increasing novelty," *Proc. Natl. Acad. Sci. USA*, vol. 120, no. 12, art. no. e2214840120, 2023.

[9] VEX Robotics, "VEX V5 Robotics Competition 2026-2027: Override game manual," v2.1, [Online]. Available: `https://manuals.vex.com/v5/override/v2.1/game/`, accessed Oct. 2026.

[10] VEX Robotics, "RobotEvents API v2," [Online]. Available: `https://www.robotevents.com/api/v2`, accessed Oct. 2026.

[11] S. Liu, Z. Zeng, T. Ren et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. ECCV*, 2024.

[12] N. Ravi, V. Gabeur, Y.-T. Hu et al., "SAM 2: Segment anything in images and videos," in *Proc. ICLR*, 2025.

[13] N. Carion, L. Gustafson, Y.-T. Hu et al., "SAM 3: Segment anything with concepts," in *Proc. ICLR*, 2026.

[14] F. Magera, T. Hoyoux, O. Barnich et al., "BroadTrack: Broadcast camera tracking for soccer," in *Proc. WACV*, 2025.

[15] H.-W. Huang, C.-Y. Yang, J. Sun et al., "Iterative scale-up ExpansionIoU and deep features association for multi-object tracking in sports," in *Proc. WACV Workshops*, 2024.

[16] B. Baker, I. Akkaya, P. Zhokhov et al., "Video PreTraining (VPT): Learning to act by watching unlabeled online videos," in *Proc. NeurIPS*, 2022.

[17] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, 1960.

[18] M. Torne Villasevil, A. Simeonov, Z. Li et al., "Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation," in *Proc. RSS*, 2024.

[19] A. Mandlekar, S. Nasiriany, B. Wen et al., "MimicGen: A data generation system for scalable robot learning using human demonstrations," in *Proc. CoRL*, 2023.

[20] G. Capellera, L. Ferraz, A. Rubio et al., "JointDiff: Bridging continuous and discrete in multi-agent trajectory generation," in *Proc. ICLR*, 2026.

[21] O. Vinyals, I. Babuschkin, W. M. Czarnecki et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, 2019.

[22] A. Bakhtin, D. J. Wu, A. Lerer et al., "Mastering the game of no-press Diplomacy via human-regularized reinforcement learning and planning," in *Proc. ICLR*, 2023.

[23] Z. Tang, C. Yu, B. Chen et al., "Discovering diverse multi-agent strategic behavior via reward randomization," in *Proc. ICLR*, 2021.

[24] M. Jaderberg, V. Dalibard, S. Osindero et al., "Population based training of neural networks," arXiv:1711.09846, 2017.

[25] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," arXiv:1504.04909, 2015.

[26] K. Greff, F. Belletti, L. Beyer et al., "Kubric: A scalable dataset generator," in *Proc. CVPR*, 2022.

[27] J. Tobin, R. Fong, A. Ray et al., "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IROS*, 2017.

[28] J. Tremblay, A. Prakash, D. Acuna et al., "Training deep networks with synthetic data: Bridging the reality gap by domain randomization," in *Proc. CVPR Workshops*, 2018.

[29] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in *Proc. IEEE ICRA*, 2011.

[30] Statbotics, "The EPA model," [Online]. Available: `https://www.statbotics.io/blog/epa`, accessed Oct. 2026.

[31] S. Ramos, S. Girgin, L. Hussenot et al., "RLDS: An ecosystem to generate, share and use datasets in reinforcement learning," arXiv:2111.02767, 2021.

[32] R. Cadene, S. Alibert, A. Soare et al., "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," 2024. [Online]. Available: `https://github.com/huggingface/lerobot`

[33] J. K. Terry, B. Black, N. Grammel et al., "PettingZoo: Gym for multi-agent reinforcement learning," in *Proc. NeurIPS*, 2021.

[34] C. Formanek, A. Jeewa, J. Shock, and A. Pretorius, "Off-the-grid MARL: Datasets with baselines for offline multi-agent reinforcement learning," in *Proc. AAMAS*, 2023, extended abstract.

[35] REC Foundation, "RECF Achieve Pinnacle game manual," 2026, [Online]. Available: `https://recf.org/documents/2026/06/game-manual-recf-achieve-pinnacle.pdf/`, accessed Oct. 2026.

[36] N. Montali, J. Lambert, P. Mougin et al., "The Waymo Open Sim Agents Challenge," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2023.

[37] V. Somers, V. Joos, A. Cioppa et al., "SoccerNet Game State Reconstruction: End-to-end athlete tracking and identification on a minimap," in *Proc. CVPR Workshops*, 2024.

[38] P. Wen, Y. Wang, and Q. Wang, "TacSIm: A dataset and benchmark for football tactical style imitation," in *Proc. CVPR*, 2026.

[39] H. Zhang, Y. Yuan, V. Makoviychuk et al., "Learning physically simulated tennis skills from broadcast videos," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[40] J. Jang, S. Ye, Z. Lin et al., "DreamGen: Unlocking generalization in robot learning through video world models," in *Proc. CoRL*, 2025.

[41] NVIDIA, J. Bjorck, F. Castañeda et al., "GR00T N1: An open foundation model for generalist humanoid robots," arXiv:2503.14734, 2025.

[42] "VEX Match Jumper," [Online]. Available: `https://jumper.robostem.org/`,

accessed Oct. 2026.
[43] A. Lim, C.-T. Chiang, and J.-C. Teng, "Estimating robot strengths with application to selection of alliance members in FIRST robotics competitions," *Comput. Stat. Data Anal.*, vol. 158, art. no. 107181, 2021.
[44] F. Galbiati, R. X. Gran, B. D. Jacques et al., "An integrated framework for team formation and winner prediction in the FIRST Robotics Competition: Model, algorithm, and analysis," arXiv:2402.00031, 2024.
[45] Roboflow, "Mapping robot paths in robotics competitions with computer vision," Aug. 2024, [Online]. Available: `https://roboflow.com/blog/robot-path-mapping`
[46] J. Zietek, N. Wade, C. Roberts et al., "Pac-Man Pete: An extensible framework for building AI in VEX Robotics," arXiv:2211.14385, 2022.
[47] H. Mellmann, B. Schlotter, and P. Strobel, "Toward data driven development in RoboCup," in *RoboCup 2019: Robot World Cup XXIII*, ser. LNCS, vol. 11531, 2019.
[48] T. Ren, S. Liu, A. Zeng et al., "Grounded SAM: Assembling open-world models for diverse visual tasks," arXiv:2401.14159, 2024.
[49] T. Cheng, L. Song, Y. Ge et al., "YOLO-World: Real-time open-vocabulary object detection," in *Proc. CVPR*, 2024.
[50] Y. Zhang, P. Sun, Y. Jiang et al., "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. ECCV*, 2022.
[51] C.-Y. Yang, H.-W. Huang, W. Chai et al., "SAMURAI: Adapting Segment Anything Model for zero-shot visual tracking with motion-aware memory," arXiv:2411.11922, 2024.
[52] V. Somers, B. Standaert, V. Joos et al., "CAMELTrack: Context-aware multi-cue ExpLoitation for online multi-object tracking," arXiv:2505.01257, 2025.
[53] J. Theiner and R. Ewerth, "TVCalib: Camera calibration for sports field registration in soccer," in *Proc. WACV*, 2023.
[54] M. Gutiérrez-Pérez and A. Agudo, "PnLCalib: Sports field registration via points and lines optimization," *Comput. Vis. Image Underst.*, vol. 267, art. no. 104712, 2026.
[55] M. Mathieu, S. Ozair, S. Srinivasan et al., "AlphaStar Unplugged: Large-scale offline reinforcement learning," arXiv:2308.03526, 2023.
[56] J. Song, H. Ren, D. Sadigh et al., "Multi-agent generative adversarial imitation learning," in *Proc. NeurIPS*, 2018.
[57] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Am. Stat. Assoc.*, vol. 22, no. 158, 1927.